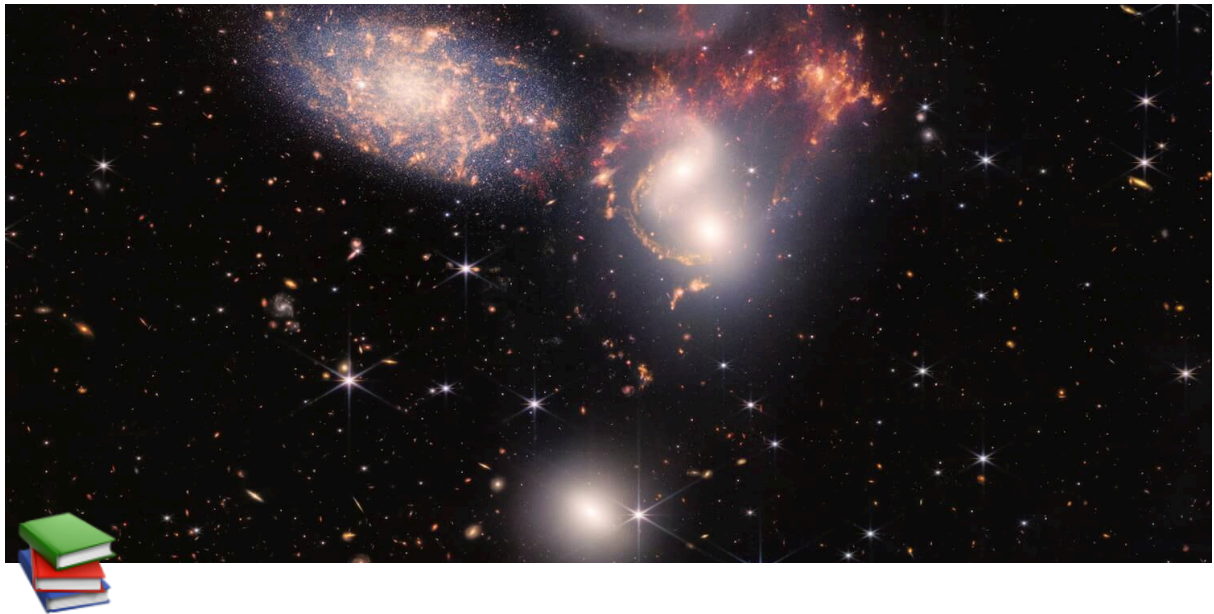




CH 5. 회귀

01 회귀 소개

<회귀 분석>

: 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법

<회귀>

여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법

독립변수 : 피쳐

종속변수 : 결정 값

회귀 계수(Regression coefficients)

★ 머신러닝 회귀 예측의 핵심

: 주어진 피쳐와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수를 찾아내는 것

<회귀 유형 분석>

독립변수 개수	회귀 계수의 결합
1개: 단일 회귀	선형: 선형 회귀
여러 개: 다중 회귀	비선형: 비선형 회귀

<분류 & 회귀>

지도학습 - 분류 & 회귀

분류 : 예측값이 카테고리나 같은 이산형 클래스 값

회귀 : 연속형 숫자 값

✓ 선형 회귀

: 실제 값과 예측값의 차이(오류의 제곱 값)을 최소화하는 직선형 회귀선을 최적화하는 방식.

- 규제(Regularization)방법에 따라 다시 별도의 유형으로 나뉠 수 있음
 - 일반적인 선형 회귀의 과적합 문제를 해결하기 위해 회귀 계수에 패널티 값을 적용하는 것.

<대표적인 선형 회귀 모델>

일반 선형 회귀: 예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화할 수 있도록 회귀 계수를 최적화하며, 규제(Regularization)를 적용하지 않은 모델입니다.

릿지(Ridge): 릿지 회귀는 선형 회귀에 L2 규제를 추가한 회귀 모델입니다. 릿지 회귀는 L2 규제를 적용하는데, L2 규제는 상대적으로 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해서 회귀 계수값을 더 작게 만드는 규제 모델입니다.

라쏘(Lasso): 라쏘 회귀는 선형 회귀에 L1 규제를 적용한 방식입니다. L2 규제가 회귀 계수 값의 크기를 줄이는 데 반해, L1 규제는 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 회귀 예측 시 피처가 선택되지 않게 하는 것입니다. 이러한 특성 때문에 L1 규제는 피처 선택 기능으로도 불립니다.

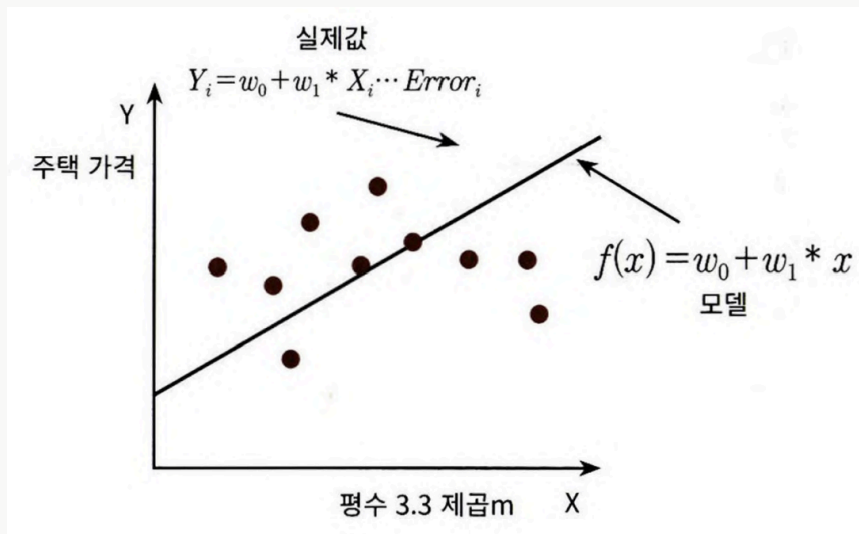
엘라스틱넷(ElasticNet): L2, L1 규제를 함께 결합한 모델입니다. 주로 피처가 많은 데이터 세트에서 적용되며, L1 규제로 피처의 개수를 줄임과 동시에 L2 규제로 계수 값의 크기를 조정합니다.

로지스틱 회귀(Logistic Regression): 로지스틱 회귀는 회귀라는 이름이 붙어 있지만, 사실은 분류에 사용되는 선형 모델입니다. 로지스틱 회귀는 매우 강력한 분류 알고리즘입니다. 일반적으로 이진 분류뿐만 아니라 희소 영역의 분류, 예를 들어 텍스트 분류와 같은 영역에서 뛰어난 예측 성능을 보입니다.

02 단순 선형 회귀를 통한 회귀 이해

단순 선형 회귀: 독립변수 1개, 종속변수 1개인 선형 회귀

Ex) 주택 가격 & 주택 크기 관계



X축: 주택의 크기 축(평당 크기) // Y축: 주택의 가격 축

주택 가격: 특정 기울기와 절편을 가진 1차 함수식으로 모델링

예측값

$$\hat{Y} = w_0 + w_1 * X$$

단순 선형 회귀 - w_1 (기울기) & w_0 (절편 intercept) : 회귀 계수

실제 주택 가격: (1차 함수 값) +/- (오류 값)

$$(w_0 + w_1 * X + \text{오류 값})$$

오류 값: 남은 오류 / 잔차

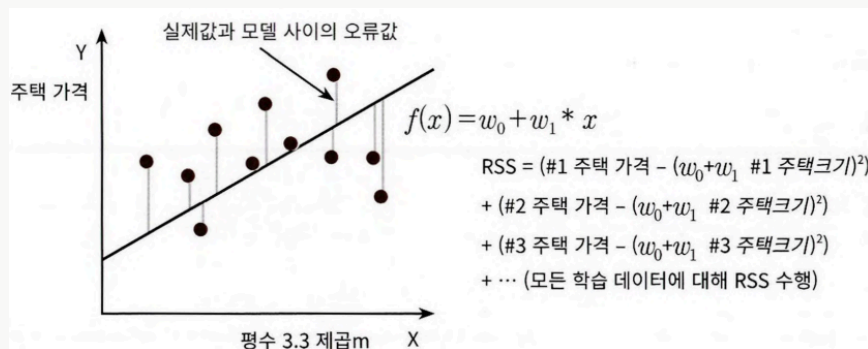
★ 최적의 회귀 모델 생성: 잔차 합이 최소가 되는 모델 생성 & 오류 값 합이 최소가 되는 최적의 회귀 계수 찾는 것!

오류 값은 + or -

→ 오류 합을 계산할 때, 절댓값을 취해 더하거나(**Mean Absolute Error**), 오류 값의 제곱을 구해서 더하는 방식(**RSS, Residual Sum of Square**)

→ 미분 등의 계산을 편리하게 하기 위해 RSS 방식으로 오류 합 구함

$$Error^2 = RSS$$



✓ 머신러닝 기반 회귀: **RSS를 최소화** 하는 **w0, w1(회귀 계수)**를 학습을 통해 찾는 것.

✓ w 변수(회귀 계수)가 중심 변수!

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

(i는 1부터 학습 데이터의 총 건수 N까지)

✓ **RSS : Cost function(Loss function)**

→ 계속 학습하면서 비용 함수가 반환하는 값(오류 값)을 감소시키고, 최소의 오류 값 구하기!

03 비용 최소화하기-경사 하강법(Gradient Discent)소개

W 파라미터 개수 적으면 → 고차원 방정식으로 비용 함수가 최소가 되는 W 변숫값을 도출

W 파라미터 많으면 → 고차원 방정식으로도 해결 어려움!

- 경사 하강법

: 고차원 방정식에 대한 문제 해결, 비용 함수 RSS 최소화하는 방법을 직관적으로 제공

: '데이터를 기반으로 알고리즘이 스스로 학습한다.'

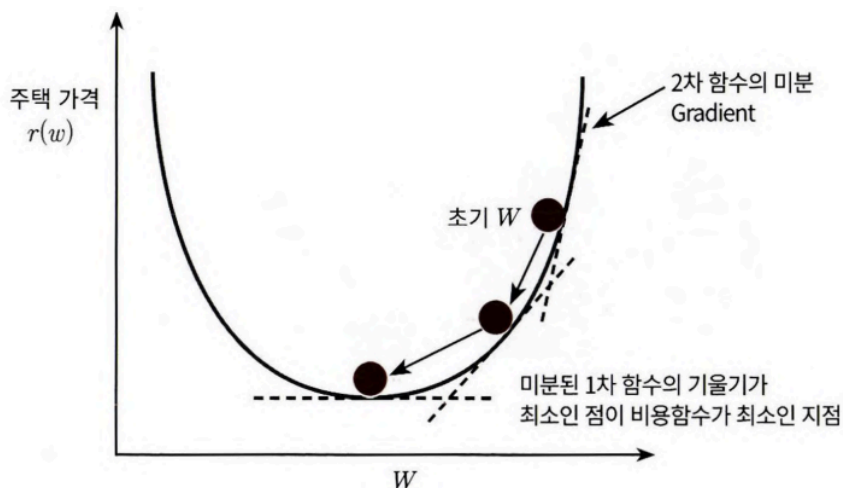
: '점진적으로' 반복적인 계산을 통해 W 파라미터 값 업데이트 → 오류 값이 최소가 되는 W 파라미터 구함

- 경사 하강법의 핵심

: "어떻게 하면 오류가 작아지는 방향으로 W값을 보정할 수 있을까?"

만약 비용 함수가 2차 함수라면,

→ 최초 w에서부터 미분을 적용 → 미분 값이 감소하는 방향으로 순차적으로 w 업데이트 → 더 이상 미분된 1차 함수의 기울기가 감소하지 않는 지점 = 비용 함수가 최소인 지점 → w 반환



- 경사 하강법의 수식 & 파이썬 코드 구현

$$R(w) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

R(w) : w0, w1 파라미터 두 개 존재 → 각 변수에 **편미분** 적용

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

w1, w0의 편미분 결과값을 마이너스(-)를 적용하며 업데이트 → 비용 함수(R(w))가 최소가 되는 w1, w0 값 도출

learning rate(학습률) : 편미분 값을 클 것을 대비해 곱하는 보정 계수

Step 1: w_1, w_0 를 임의의 값으로 설정하고 첫 비용 함수의 값을 계산합니다.

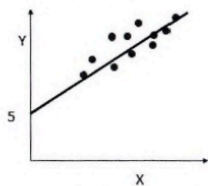
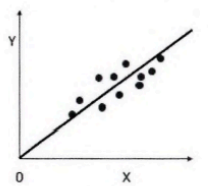
Step 2: w_1 을 $w_1 + \eta \frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$, w_0 을 $w_0 + \eta \frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$ 으로 업데이트한 후 다시 비용 함수의 값을 계산합니다.

Step 3: 비용 함수가 감소하는 방향으로 주어진 횟수만큼 Step 2를 반복하면서 w_1 과 w_0 를 계속 업데이트합니다.

04 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

- **LinearRegression 클래스 - Ordinary Least Squares**

: 예측값과 실제 값의 RSS(Residual Sum of Squares)를 최소화해 OLS(Ordinary Least Squares) 추정 방식으로 구현한 클래스

입력 파라미터	fit_intercept: 불린 값으로, 디폴트는 True입니다. Intercept(절편) 값을 계산할 것인지 말지를 지정합니다. 만일 False로 지정하면 intercept가 사용되지 않고 0으로 지정됩니다. fit_intercept=True  fit_intercept=False 
	normalize: 불린 값으로 디폴트는 False입니다. fit_intercept가 False인 경우에는 이 파라미터가 무시됩니다. 만일 True이면 회귀를 수행하기 전에 입력 데이터 세트를 정규화합니다.
속성	coef_: fit() 메서드를 수행했을 때 회귀 계수가 배열 형태로 저장하는 속성. Shape는 (Target 값 개수, 피쳐 개수). intercept_: intercept 값

다중 공선성 문제 - PCA 통해 차원 축소!

• 회귀 평가 지표

회귀의 평가를 위한 지표 - 실제 값 & 회귀 예측값의 차이 값 기반 지표

실제 값 & 예측값 차이를 더하면 + & - 섞여서 오류 상쇄

평가 지표	설명	수식
MAE	Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다.	$MAE = \frac{1}{n} \sum_{i=1}^n Y_i - \hat{Y}_i $
MSE	Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.	$MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$
RMSE	MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다.	$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$
R ²	분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.	$R^2 = \frac{\text{예측값 Variance}}{\text{실제값 Variance}}$

평가 방법	사이킷런 평가 지표 API	Scoring 함수 적용 값
MAE	metrics.mean_absolute_error	'neg_mean_absolute_error'
MSE	metrics.mean_squared_error	'neg_mean_squared_error'
RMSE	metrics.mean_squared_error를 그대로 사용하되 squared 파라미터를 False로 설정.	'neg_root_mean_squared_error'
MSLE	metrics.mean_squared_log_error	'neg_mean_squared_log_error'
R ²	metrics.r2_score	'r2'

실제 값과 예측값의 오류 차이를 기반으로 하는 회귀 평가 지표

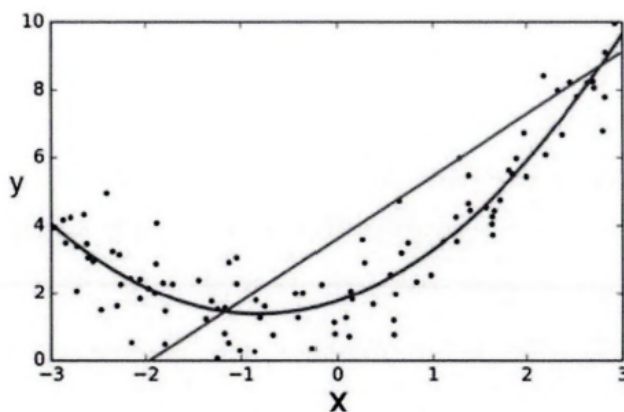
—> 값이 커지면 나쁜 모델이라는 의미 ⇒ 보정 필요

05 다항 회귀와 과(대)적합/과소적합 이해

- 다항 회귀 이해

회귀가 독립변수의 다항식이 아닌 2차, 3차 방정식과 같은 다항식으로 표현되는 것을 다항 (Polynomial) 회귀라고 함

다항 회귀 - 선형 회귀



〈 주어진 데이터 세트에서 다항 회귀가 더 효과적임 〉

- 다항 회귀를 이용한 과소적합 및 과적합 이해

다항 회귀: 피처의 직선적 관계가 아닌 복잡한 다항 관계를 모델링

다항식의 차수 - 높아질수록 매우 복잡한 피처 간의 관계까지 모델링 가능

But, 차수가 높아질수록 과적합의 문제 크게 발생

맨 왼쪽의 Degree 1 예측 곡선은 단순한 직선으로서 단순 선형 회귀와 똑같습니다. 실제 데이터 세트인 코사인 데이터 세트를 직선으로 예측하기에는 너무 단순해 보입니다. 예측 곡선이 학습 데이터의 패턴을 제대로 반영하지 못하고 있는 과소적합 모델이 되었습니다. MSE 값은 약 0.41입니다.

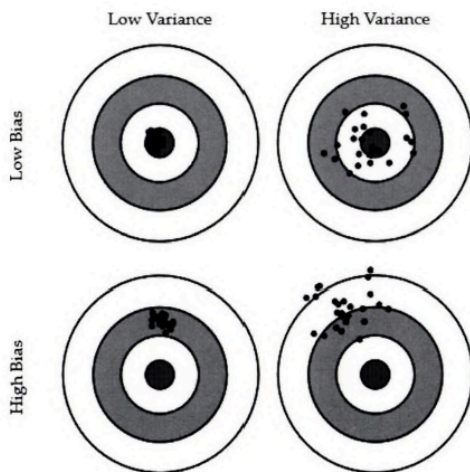
가운데 Degree 4 예측 곡선은 실제 데이터 세트와 유사한 모습입니다. 변동하는 잡음까지 예측하지는 못했지만, 학습 데이터 세트를 비교적 잘 반영해 코사인 곡선 기반으로 테스트 데이터를 잘 예측한 곡선을 가진 모델이 되었습니다. MSE 값은 약 0.04로 가장 뛰어난 예측 성능을 나타내고 있습니다.

맨 오른쪽 Degree 15 예측 곡선은 MSE 값이 182581084.83이 될 정도로 어처구니없는 오류 값이 발생했습니다(물론 과적합을 강조하기 위해 Degree를 매우 높은 차수인 15로 설정한 결과입니다). 예측 곡선을 보면 데이터 세트의 변동 잡음 값까지 지나치게 반영한 결과, 예측 곡선이 학습 데이터 세트만 정확히 예측하고, 테스트 값의 실제 곡선과는 완전히 다른 형태의 예측 곡선이 만들어졌습니다. 결과적으로 학습 데이터에 너무 충실하게 맞춘 과적합이 심한 모델이 되었고 어이없는 수준의 높은 MSE 값이 나왔습니다.

• 편향-분산 트레이드오프(Bias-Variance Trade off)

고편향(High Bias) : 지나치게 한 방향으로 치우친 경향

고분산(High Variance) : 지나치게 높은 변동성



〈 편향과 분산의 고/저에 따른 표현.

<http://scott.fortmann-roe.com/docs/BiasVariance.html>에서 발췌 >

편향이 높으면 → 분산은 낮아짐 / 편향이 낮으면 → 분산은 높아짐

높은 편향/낮은 분산에서 과소적합되기 쉬움

낮은 편향/높은 분산에서 과적합되기 쉬움

★ 편향과 분산이 서로 트레이드오프 & 오류 Cost 값이 최대한 낮아지는 모델 구축

06 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

• 규제 선형 모델의 개요

비용 함수- RSS를 최소화/ 실제 값과 예측값의 차이를 최소화하는 것만 고려

→ 학습 데이터에 지나치게 맞추게 되고, 회귀 계수가 쉽게 커짐

→ 변동성이 오히려 심해져서 테스트 데이터 세트에서는 예측 성능이 저하됨

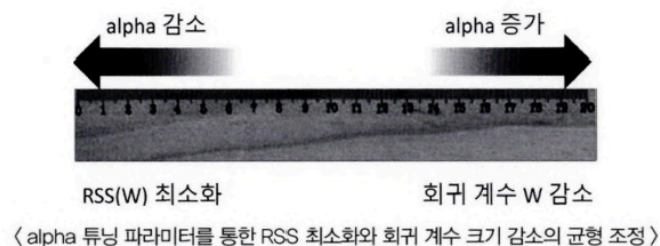
✓ 비용 함수- 학습 데이터의 잔차 오류 값을 최소로 하는 RSS 최소화 방법과 과적합을 방지하기 위해

회귀 계수 값이 커지지 않도록 하는 방법이 서로 균형



$$\text{비용 함수 목표} = \text{Min}(\text{RSS}(W) + \alpha * \|W\|_2^2)$$

- $\alpha = 0$ 인 경우는 W 가 커도 $\alpha * \|W\|_2^2$ 가 0이 되어 비용 함수는 $\text{Min}(\text{RSS}(W))$
- $\alpha = \text{무한대}$ 인 경우 $\alpha * \|W\|_2^2$ 도 무한대가 되므로 비용 함수는 W 를 0에 가깝게 최소화 해야 함.



• 릿지 회귀

Ridge 클래스의 주요 생성 파라미터: α (릿지 회귀의 α L2 규제 계수)

• 라쏘 회귀

W의 절댓값에 페널티를 부여하는 L1 규제를 선형 회귀에 적용한 것: 라쏘(Lasso) 회귀
Lasso 클래스의 주요 생성 파라미터: alpha(라쏘 회귀의 alpha L1 규제 계수)

• 엘라스틱넷 회귀

엘라스틱넷(Elastic Net) 회귀: L2 규제 + L1 규제

라쏘 회귀가 서로 상관관계가 높은 피쳐들 중에서 중요 피쳐만 선택하고, 다른 피쳐들은 모두 회귀 계수를 0으로 만드는 성향 강함

단점: L1과 L2 규제가 결합된 규제로 인해 수행시간이 상대적으로 오래 걸림

• 선형 회귀 모델을 위한 데이터 변환

피쳐와 타깃값 간에 선형의 관계가 있다고 가정 → 최적의 선형함수를 찾아 결괏값 예측

피쳐값과 타깃값의 분포: 정규 분포(즉 평균을 중심으로 종 모양으로 데이터 값이 분포된 형태) 형태 선호

선형 회귀 모델 적용하기 전에 데이터에 대한 스케일링/정규화 작업을 수행

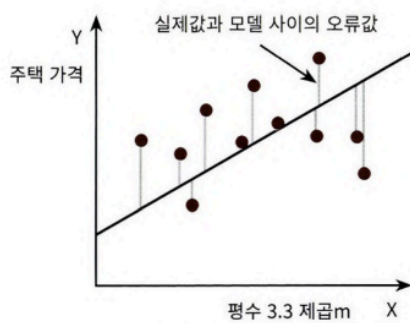
1. StandardScaler 클래스를 이용해 평균이 0, 분산이 1인 표준 정규 분포를 가진 데이터 세트로 변환하거나 MinMaxScaler 클래스를 이용해 최솟값이 0이고 최댓값이 1인 값으로 정규화를 수행합니다.
2. 스케일링/정규화를 수행한 데이터 세트에 다시 다항 특성을 적용하여 변환하는 방법입니다. 보통 1번 방법을 통해 예측 성능에 향상이 없을 경우 이와 같은 방법을 적용합니다.
3. 원래 값에 log 함수를 적용하면 보다 정규 분포에 가까운 형태로 값이 분포됩니다. 이러한 변환을 로그 변환(Log Transformation)이라고 부릅니다. 로그 변환은 매우 유용한 변환이며, 실제로 선형 회귀에서는 앞에서 소개한 1, 2번 방법보다 로그 변환이 훨씬 많이 사용되는 변환 방법입니다. 왜냐하면 1번 방법의 경우 예측 성능 향상을 크게 기대하기 어려운 경우가 많으며 2번 방법의 경우 피쳐의 개수가 매우 많을 경우에는 다항 변환으로 생성되는 피쳐의 개수가 기하급수로 늘어나서 과적합의 이슈가 발생할 수 있기 때문입니다.

변환 유형	alpha 값			
	alpha=0.1	alpha=1	alpha=10	alpha=100
원본 데이터	5.788	5.653	5.518	5.330
표준 정규 분포	5.826	5.803	5.637	5.421
표준 정규 분포 + 2차 다항식	8.827	6.871	5.485	4.634
최솟값/최댓값 정규화	5.764	5.465	5.754	7.635
최솟값/최댓값 정규화 + 2차 다항식	5.298	4.323	5.185	6.538
로그 변환	4.770	4.676	4.836	6.241

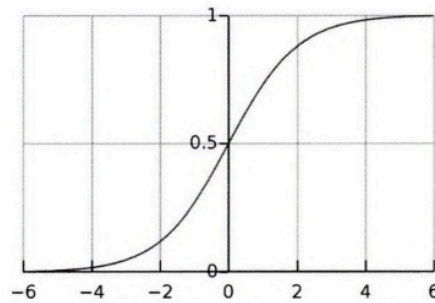
07 로지스틱 회귀

로지스틱 회귀: 선형 회귀 방식을 분류에 적용한 알고리즘

시그모이드(Sigmoid) 함수 최적선을 찾고, 반환 값을 확률로 간주해 확률에 따라 분류를 결정

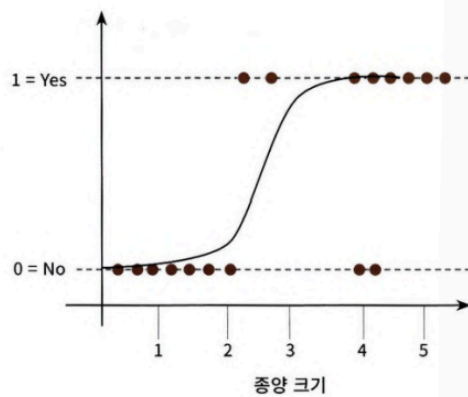
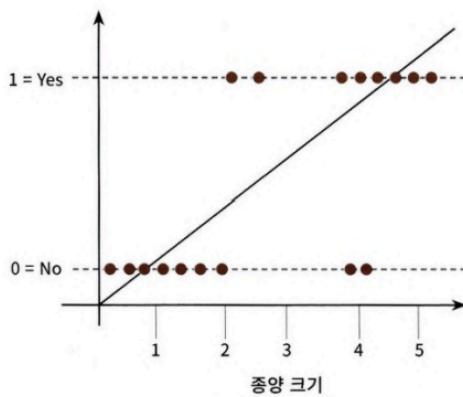


〈 선형 회귀의 선형 함수 〉



〈 로지스틱 회귀의 시그모이드 함수 〉

$$y = \frac{1}{1 + e^{-x}}$$



lbfgs: 사이킷런 버전 0.22부터 solver의 기본 설정값입니다. 메모리 공간을 절약할 수 있고, CPU 코어 수가 많다면 최적화를 병렬로 수행할 수 있습니다.

liblinear: 사이킷런 버전 0.21까지에서 solver의 기본 설정값입니다. 다차원이고 작은 데이터 세트에서 효과적으로 동작하지만 국소 최적화(Local Minimum)에 이슈가 있고, 병렬로 최적화할 수 없습니다.

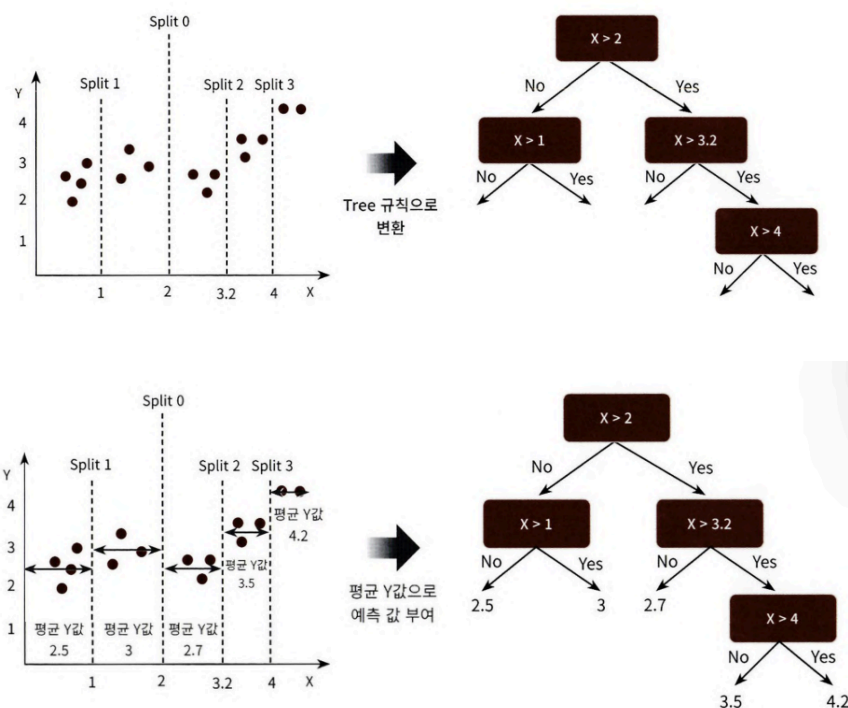
newton-cg: 좀 더 정교한 최적화를 가능하게 하지만, 대용량의 데이터에서 속도가 많이 느려집니다.

sag: Stochastic Average Gradient로서 경사 하강법 기반의 최적화를 적용합니다. 대용량의 데이터에서 빠르게 최적화합니다.

saga: sag와 유사한 최적화 방식이며 L1 정규화를 가능하게 해줍니다.

08 회귀 트리

트리 기반의 회귀: 회귀를 위한 트리를 생성하고 이를 기반으로 회귀 예측



결정 트리, 랜덤 포레스트, GBM, XGBoost, LightGBM 등의 모든 트리 기반의 알고리즘: 분류뿐만 아니라 회귀도 가능

알고리즘	회귀 Estimator 클래스	분류 Estimator 클래스
Decision Tree	DecisionTreeRegressor	DecisionTreeClassifier
Gradient Boosting	GradientBoostingRegressor	GradientBoostingClassifier
XGBoost	XGBRegressor	XGBClassifier
LightGBM	LightGBMRegressor	LightGBMClassifier